Probe-Guided Rebalancing of Persistent Subscription Connections with Greedy and Deep Q-Learning Control

MANISH KUMAR AGRAWAL, Indian Institute of Technology Roorkee, India
SANDEEP KUMAR, Indian Institute of Technology Roorkee, India

Transport load balancers cannot observe the application-level signals that affect persistent subscription delivery. These include the subscriptions on each connection, backlog per subscription, client acknowledgement capacity, and pod reliability under subscription pressure. As a result, balanced connection counts can still hide large differences in subscription counts and queue backlog. We model client-side rebalancing as a sequence of decisions based on partial observations. We present a probe-guided rebalancing framework for persistent subscription transports, implemented and tested with STOMP. A client combines telemetry from active persistent connections, local queue and throughput counters, and one subscription-free connection that the existing load balancer assigns to a random pod. The framework separates policy selection from receipt-gated execution. It supports deterministic, pressure-aware greedy control and Deep Q-Learning (DQL) over bounded actions. Both use subscription count and backlog to assess a candidate; neither needs full cluster visibility. The DQL policy additionally learns the tradeoff among backlog, imbalance, throughput, and intervention cost. Execution guards block infeasible actions. When migration is selected, the probe is promoted through a receipt-gated standby, quiesce, drain, activate, detach, and old-socket retirement protocol.

In a locked scale-out simulator experiment, 20,000 subscriptions carry 775 million queued events on ten connections established on one Pod. Across 20 paired seeds, adding three Pods with unchanged connection placement yields 58,000 events/s. Probe-guided DQL and guarded-greedy migration yield 219,762 and 221,260 events/s, respectively. DQL reduces normalized backlog area by 12.30% relative to unchanged placement (paired difference -0.117504; 95% confidence interval $[-0.121210, -0.113345]$). The comparison establishes the value of connection reassociation under the stated service model. In a separate mixed-workload holdout, DQL improves mean composite reward by 0.0318 and reduces subscription-count variance by 90.0% relative to the two-dimensional heuristic, with a 4.5% throughput tradeoff. Oracle Cloud experiments establish execution of the mechanism: ten connection replacements reassociate 4,000 of 20,000 subscriptions, and a loaded scale-out trial completes three selective handoffs while retaining the other sessions and confirming all 8,000 observed delivery ACKs. The framework therefore offers both a transparent greedy policy and a learned multi-objective policy, with their benefits evaluated separately from the shared handoff mechanism.

CCS Concepts: • **Computing methodologies** → **Reinforcement learning**; • **Computer systems organization** → **Cloud computing**; • **Software and its engineering** → *Message passing*; *Publish-subscribe / event-based architectures.*

Additional Key Words and Phrases: persistent connections, subscription rebalancing, deep Q-learning, partial observability, probe promotion, greedy control, messaging systems, autonomic control

1 Introduction

Distributed messaging systems support cloud-native and event-driven applications. Real-time analytics, notifications, workflow engines, and microservices need reliable event delivery as workloads change. Many deployments run several subscriber-service instances behind a transport-layer load balancer. Clients keep persistent STOMP/TCP, raw TCP, or WebSocket-style connections to these services.

Authors' Contact Information: Manish Kumar Agrawal, m_agrawal@cs.iitr.ac.in, Department of Computer Science and Engineering, Indian Institute of Technology Roorkee, Roorkee, Uttarakhand, India; Sandeep Kumar, sandeep.garg@cs.iitr.ac.in, Department of Computer Science and Engineering, Indian Institute of Technology Roorkee, Roorkee, Uttarakhand, India.

Load balancing is therefore an important design concern. Poor distribution leaves some subscriber instances with growing backlogs and others with spare capacity. Clients then experience uneven delivery delays. Cloud resource-management reviews describe machine-learning methods for changing demand and capacity [25]. Deep-reinforcement-learning reviews also discuss adaptive policies for different resource types and competing objectives [42]. Conventional load balancers, however, usually distribute connections through policies such as round-robin or least-connections.

Those policies are simple and scalable, but connection count alone does not describe subscription-heavy messaging. A persistent connection may carry a few lightweight subscriptions or hundreds of high-rate subscriptions with growing queues. The load balancer may count both cases as one connection. The subscriber pod handles very different workloads in these cases. Its load depends on subscription count, backlog per subscription, and the client's message-processing and acknowledgement rates.

Subscriber-pod scale-out exposes this mismatch directly. If ten persistent connections are established while only one pod exists, new pods increase available processing capacity but do not change those established endpoints. The client can use the new capacity by promoting a probe that reaches a suitable pod and reassociating subscription-bearing connections to it. This scale-out recovery is the central experiment in Section 7.1. The corresponding live handoff mechanism is examined during loaded Oracle Cloud cluster scale-out in Section 6.4.

This mismatch creates two related problems. First, a client may reach a pod that already manages too many subscriptions, even when its connection count is acceptable. Second, the client may place too many subscriptions on one of its own connections. This can create local backlog even when the pod has enough capacity overall. A fixed connection pool cannot increase parallelism as subscription counts grow or event rates rise. Opening more connections without moving subscriptions, however, can leave the workload unevenly distributed.

Deep reinforcement learning has been used for hierarchical cloud scheduling [43], multidimensional data-center allocation [38], and distributed cloud-edge offloading [31]. Decentralized bandit methods select service instances from client QoS feedback [12] or coordinate edge offloading through server rewards [40]. BLOC also shows how client-side load balancing can use overload feedback without a central metadata store [2]. These studies provide examples of adaptive control with limited information. Our focus is how the client changes persistent subscription connections. It retains a probed socket, promotes it only when the required checks pass, and transfers subscription ownership while preserving the durable queues.

Server placement, connection scaling, and subscription assignment depend on each other. Placement determines which pod a connection reaches. Scaling determines how many connections the client can use in parallel. Subscription assignment determines how evenly the work is spread across those connections. A practical controller must coordinate these decisions using only the information available to the client at runtime.

We propose an adaptive framework with coordinated control loops for persistent subscription messaging. It applies to long-lived STOMP/TCP, raw TCP, or WebSocket connections that carry multiple subscriptions and periodically report application-level statistics. The framework has three control loops at different system layers:

(1) A policy layer that chooses bounded actions from active-connection and probe statistics. We evaluate deterministic greedy rules and DQL as alternative selectors under partial observability.
(2) A client-side adaptive parallel connection scaling loop that adjusts the number of persistent transport connections based on total subscription count and per-subscription backlog characteristics.
(3) A connection-level load rebalancing loop that minimizes both subscription-count variance and per-subscription-backlog variance across scaled connections.

Together, these loops let the client choose and carry out rebalancing actions without full cluster visibility. Both policy families use the client's own queues, active-connection statistics, and one retained probe assigned by the load balancer. The greedy policy responds to measured pressure and imbalance. DQL learns action values for the same operational objectives. The design keeps the existing transport load balancer.

Contributions

The key contributions of this work are:

- A client-side connection-replacement mechanism for persistent subscription transports. The client retains an authenticated, load-balancer-assigned probe and conditionally promotes that same socket, preserving the observed endpoint without requiring direct routing to a chosen Pod.
- A policy-independent separation of observation, action selection, and guarded execution. A pressure-aware greedy policy and a partially observable DQL policy use client-local queues and sampled Pod statistics. The experiments distinguish the benefit of the common mechanism from the choice of selector.
- Subscription- and backlog-aware connection scaling and two-dimensional connection rebalancing. These mechanisms distinguish adding transport parallelism, redistributing subscriptions across existing connections, and replacing a connection's delivery endpoint.
- An implemented receipt-gated handoff that retains subscription identity and durable queues. Standby target bindings, source quiescence, a fresh zero-in-flight check, target activation, and source retirement make migration an explicit protocol rather than an instantaneous reassignment.
- Complementary deployment and policy evidence: a 20,000-subscription Oracle Cloud cluster connection-replacement experiment, an isolated loaded scale-out trial with selective DQL handoffs, and controlled simulator comparisons and ablations. The evaluation separates executable handoff behavior, the benefit of rebalancing, and the contribution of learned action selection.

The contribution is not a new Q-learning update or a new greedy optimizer. It is their integration with retained-probe selection and subscription-aware connection replacement, supported by separate mechanism and policy tests. Greedy control provides a transparent reference and a usable control policy; DQL provides a learned choice among competing operational objectives. The reported experiments evaluate them separately. They do not use a DQL model to tune the scripted comparator or claim an evaluated hybrid policy.

2 Background and Related Work

2.1 Persistent Messaging and Stream Processing

We consider persistent subscription transports in which a client keeps a long-lived connection to a subscriber service and multiplexes multiple subscriptions over that connection. The abstraction covers STOMP-over-TCP, custom TCP subscription streams, and WebSocket-based messaging. In each case, the load created by one connection is not captured by the connection count alone; it also depends on how many subscriptions are multiplexed over the connection and how much backlog has accumulated behind those subscriptions.

Modern event-driven platforms and stream-processing engines face related elasticity problems. Dynamic stream redirection can improve runtime elasticity in distributed stream-processing systems [36]. Cardellini et al. survey runtime-adaptation strategies and organize them by their control objectives and mechanisms [4]. Recent adaptive join-processing work also shows that workload redistribution can improve latency and throughput under changing

stream rates [37]. These studies motivate adaptive runtime control. Their actions change operator placement, stream routing, or stateful processing inside the stream runtime. They do not address the client-side persistent-transport problem in which subscription count and per-subscription backlog must be balanced across a client's own long-lived connections.

Fragkoulis et al. distinguish elasticity, reconfiguration, state management, and fault tolerance in the evolution of stream-processing systems [15]. This distinction matters here because the durable queue stays in place: migration changes its subscription's delivery owner, not the queue's stored contents. Transactional stream processing also separates processing guarantees from ordinary stream execution [41]. Accordingly, a completed connection replacement is not, by itself, evidence of end-to-end exactly-once processing.

2.2 Resource Allocation and Scheduling

Resource allocation offers relevant control designs, but acts on different objects. Chen et al. combine predicted Q-values with feedback control to allocate cloud-service resources [9]. Their workload-window approach accounts for current and future demand [8]. Actor–critic allocation addresses dynamic resource assignment in datacenters [10]. Caviglione et al. instead use DRL to choose among VM-placement heuristics with several objectives [6]. This is relevant to separating action selection from execution. Here, the executed action changes a subscription binding or a persistent connection, rather than VM placement.

Task schedulers also combine resource pressure with service objectives. Ding et al. separate cloud-level task dispatch from per-server Q-learning scheduling [13]. Chen et al. coordinate heterogeneous workflows subject to task-continuity constraints [7]. Li et al. use DRL for energy-aware Spark executor allocation under execution-time constraints [27]. Cheng et al. study preemptive scheduling to reduce cloud-job cost [11], while Liu et al. include inter-region bandwidth and transfer cost in data-intensive task placement [28]. The review by Sanjalawe et al. places these scheduling objectives within the wider AI-based cloud literature [33]. Our comparison concerns a different execution requirement: preserving the evaluated transport endpoint while transferring delivery ownership between existing client sessions.

Edge offloading provides related examples of constrained action selection. Hortelano et al. classify RL approaches by their architectures, objectives, and changing operating conditions [19]. Huang et al. learn binary offloading choices for wireless-powered devices [21]. Jayanetti et al. separate edge/cloud-tier selection from node selection for precedence-constrained tasks [23]. These studies help distinguish the decision space from the physical operation it controls. In this paper, bounded templates select subscription redistribution or retained-probe promotion, and receipt checks govern the resulting handoff.

2.3 Autonomic and Learning-Based Control

Gheibi et al. organize machine-learning support for self-adaptation around monitor–analyze–plan–execute feedback loops, including policy updates and resource management [17]. Our controller follows that established pattern; DQL alone is not the differentiating mechanism. The industrial survey by Weyns et al. also examines the practical motivations, risks, and difficulties of deploying self-adaptation [39]. This motivates making the observation boundary and the concrete connection-handoff procedure explicit.

Recent DRL literature is also clear about the risks of applying learning methods directly to production systems. Partial observability, non-stationary workloads, reward design, and safe online adaptation remain difficult [14]. Cloud DRL surveys make the same point for operational cloud control [42]. The closer load-balancing comparisons already include decentralized operation. The QEdgeProxy preprint describes multi-player bandits and per-client QoS feedback for

selecting service instances [12]. Yahya et al. combine mean-field bandits with server-reward adjustment for decentralized task offloading [40]. BLOC uses overload feedback and dynamic capacity estimation without a centralized metadata store [2]. Selvakumar et al. use predicted waiting times and service-chain communication costs to prioritize requests and select instances [35]. These mechanisms provide direct comparisons for decision inputs and adaptive selection. Our narrower distinction is conditional reuse of a retained probe socket together with a concrete subscription-ownership handoff.

Negi et al. classify soft-computing approaches to VM- and physical-machine-level load balancing [30]. In our messaging setting, a bounded action-template design limits the corresponding control space: the DQL agent chooses among deployable rebalancing actions rather than directly searching the full combinatorial subscription-assignment space.

Metzger et al. use feature models to guide exploration through large or evolving adaptation spaces [29]. Our bounded templates reduce a different search space: the possible subscription moves and connection replacements available to one client. Recent RS-DRL work instead changes reward shaping during replay to manage uncertainty in self-adaptation [24]. We retain a conventional DQL learner and specialize the observations, feasible actions, and handoff semantics to persistent messaging.

Monitoring and policy adaptation also need separate treatment. DEMon uses decentralized monitoring with adaptive parameters to balance information quality and resource use [22]. Our observation boundary is narrower: each client sees its active endpoints and one retained probe. Gheibi and Weyns study drift in the quality of available adaptation options [16]. This supports distinguishing a stored policy from the conditions under which it was evaluated. Casimiro et al. use probabilistic model checking to select adaptations of ML-based systems [5]. Our execution guards check concrete handoff conditions; they are not a model-checking guarantee for the learned policy.

2.4 Research Gap and Novelty

The comparison above identifies a specific gap in how actions are chosen and carried out. Decentralized load balancing already exists. BLOC, QEdgeProxy, and decentralized offloading already address decisions without a central full-state controller. Our mechanism combines four requirements. It treats pod subscription count and backlog as separate pressure signals. It tracks the client's subscription-to-connection assignments. It promotes the same authenticated probe socket only after checking that candidate. Finally, it uses receipt-gated delivery handoff while durable queues stay in place. These requirements define the comparison in Table 1.

The proposed mechanism connects these concerns through an endpoint-preserving probe workflow. The load balancer selects a reachable Pod when the probe is opened; the selected policy then evaluates whether to use that already-established connection. Promotion is conditional, not an automatic consequence of probing. An admissible migration reuses the observed socket and transfers subscription delivery ownership under the handoff contract, while the durable queues remain stationary. This differs from choosing a Pod by name and opening another load-balanced socket, whose endpoint could differ from the one evaluated. Local subscription and backlog measurements also support scaling and redistribution across existing connections. The contribution is this partially observable control and execution mechanism, evaluated with established greedy and Q-learning methods. Their comparison is specific to the measured workloads. Table 1 summarizes the positioning of the proposed approach relative to representative literature.

The broader cloud-scheduling and edge-computing context is retained in Section E of the Online Supplementary Material.

Table 1. Comparison of Representative Work and the Proposed Framework

Work	Main focus	Observed information	Controlled object	How this work differs
BLOC [2]	Request balancing and overload control	Overload feedback; no central metadata store	Request admission and server selection	Adds retained-socket promotion and subscription ownership transfer
QEdgeProxy [12]	QoS-aware service selection	Client QoS outcomes; decentralized bandits	Service-instance selection	Preserves the probed endpoint during connection replacement
Adaptive load balancing [35]	Service-chain latency	Predicted waiting times; inter-service costs	Request priorities and instance selection	Reassociates existing subscription-bearing connections
Decentralized offloading [40]	Task distribution among edge servers	Unknown network information; adjusted server rewards	Offloading population control	Reassociates persistent subscription groups
Per-device DRL [31]	Energy- and latency-aware offloading	Device, server, channel, and task state	Local, MEC, or cloud task execution	Uses subscription counts and queued-event pressure
Partial-observation DRL [20]	QoE-aware distributed offloading	Partial observations; device QoE preferences	IoT task-offloading decisions	Evaluates a retained probe and transfers subscriptions
Dr-Stream [36]	Stream redirection and operator elasticity	Stream-runtime measurements	Operator instances and stream state	Keeps durable queues stationary during transport handoff
Aj-Stream [37]	Skew-aware stream joining	Predicted key frequencies; instance load	Key grouping and operator scaling	Moves subscription ownership while queues stay in place
RL autoscaling [1]	Microservice scaling thresholds	Resource demand and response-time QoS	Available service capacity	Moves existing sticky sockets to available capacity
HephaestusForge [34]	Multi-cluster microservice placement	Network latency; resource capacity; deployment cost	Pod-replica placement across clusters	Connects existing subscriptions to available Pod capacity
Feature-model-guided RL [29]	Exploration of adaptation choices	Feature models; configuration changes; rewards	Runtime feature configurations	Uses bounded connection actions with receipt-gated handoff
RS-DRL [24]	Reward-shaped self-adaptation	Runtime feedback; replayed transitions	Self-adaptive IoT configurations	Specializes actions to guarded transport and subscription handoff

3 System Model and Receipt-Gated Handoff

Figure 1 shows the shared architecture. Event producers send messages through ingest services to durable subscription queues. Subscriber Pods deliver them to clients over persistent connections. The transport load balancer assigns new sockets; the client selects whether to promote its retained probe. Additional component descriptions are provided in the Online Supplementary Material.

3.1 Retained-Probe Acquisition and Receipt-Gated Handoff

Figure 1 makes the selected-Pod mechanism explicit. The client does not choose a Pod name and then attempt to bypass the Kubernetes Service. It opens an authenticated, statistics-only probe through the same transport load balancer used by ordinary connections. The load balancer selects one ready subscriber Pod when that TCP/WebSocket connection is established, after which the socket remains endpoint-bound to that Pod. The STOMP implementation identifies the authenticated session and its Pod in telemetry. A probe initially has no subscription bindings and does not fetch events. Promotion changes the role of that existing session; it does not create a second connection whose load-balanced destination might differ.

The selector evaluates a candidate that is already connected. Fresh and sufficiently complete telemetry, a distinct source and target Pod, capacity, cooldown, and handoff checks determine whether migration is admissible. The DQL selector also applies its action-value hysteresis test. A rejected or unselected probe remains statistics-only until refreshed or recycled. Thus, probing does not imply migration, and the client never moves an established TCP socket between endpoints. It replaces one delivery socket with the already-connected probe and reassociates the source subscriptions. In Figure 1, c_p labels the retained probe, whose endpoint is $i_u^{\text{probe}}(t)$. The source and existing target Pod indices are $i_h = i_{u,c_h}(t)$ and $i_l = i_{u,c_l}(t)$. Thus i_h and i_l identify Pods, while $P_i(t)$ denotes a pressure score.

Queued events remain in the same durable subscription queues $Q_{u,k}(t)$. The client retains each subscription identifier, listener, message permits, and negative-acknowledgement visibility setting. For the selected subscription set $\mathcal{K}_{\text{move}}$, the executed protocol is:

(1) Create each target binding in *standby*, waiting for its STOMP receipt. The first subscription request carries the probe-promotion flag. The server checks the authenticated session and its active-connection cap, then changes the probe's role. Standby bindings do not deliver events. The source remains active while these target bindings are prepared.
(2) After all standby receipts, add the promoted socket to the client pool and clear its probe role. Send a receipt-gated quiesce request for each source binding. Quiescence stops new delivery while existing in-flight messages remain on the source acknowledgement path.
(3) Record a barrier timestamp after the quiesce requests. Wait for a newer source telemetry frame that reports every selected subscription as quiesced with zero in-flight messages. A zero from an older frame cannot satisfy the barrier. Drain time is observed, not estimated from H_i.
(4) Activate the target bindings one at a time, waiting for their receipts. The source remains quiesced during this phase.
(5) Unsubscribe the source bindings with receipt checks. After all selected bindings are detached, remove and disconnect the old socket for a full-connection migration. Replenish the subscription-free probe asynchronously. Unselected active sockets remain connected.

Let s_b be the barrier timestamp, s_f the timestamp of a received source frame, and $I_{c_h,k}(s_f)$ its reported in-flight count. The target-activation precondition is

$$s_f \geq s_b \quad \wedge \quad \bigwedge_{k \in \mathcal{K}_{\text{move}}} \left[\text{state}_{c_h,k}(s_f) = \text{quiesced} \wedge I_{c_h,k}(s_f) = 0 \right]. \tag{1}$$

This precondition describes the implemented planned handoff, assuming timely, accurate telemetry and servers that honor quiescence. It is not a distributed ownership lease or an epoch-fencing guarantee during partitions. On an exception, the client attempts to quiesce and close the target before restoring the source; unsuccessful cleanup is

logged rather than counted as a completed handoff. The protocol uses authenticated session state and receipts, not a separate signed promotion token or an atomic cross-Pod ownership commit. Receipt completion records server acknowledgement of the corresponding operation; it is not by itself proof of an end-to-end exactly-once event census. The delivery model remains unordered and permits redelivery after an unconfirmed acknowledgement. The live results report observed deliveries, ACK receipts, and unresolved records separately.

4 Problem Formulation

The main quantities are client subscriptions $\mathcal{K}_u$, active sockets C_u, per-subscription backlog $b_{u,k}$, and the corresponding connection loads $(N_{u,c}, G_{u,c})$. Pod-wide quantities (X_i, B_i) include all clients served by Pod i. Appendix B of the Online Supplementary Material collects the full symbol reference; definitions also appear at their first use below.

We model the system at discrete control epochs $t = 0, 1, \ldots, T$. The interval between adjacent epochs is Δ seconds: the next epoch is $t + 1$, while its physical time is $(t + 1)\Delta$. Live-event timestamps and durations are reported separately in seconds. Let $\mathcal{S} = \{1, \ldots, n\}$ be the set of subscriber pods and $\mathcal{U} = \{1, \ldots, m\}$ be the set of clients. Client u maintains a subscription set $\mathcal{K}_u(t)$ and a set of persistent transport connections $C_u(t)$, with $1 \le |C_u(t)| \le C_{\max}$. Indices i, u, c, k refer to Pods, clients, connections, and subscriptions, respectively. Calligraphic symbols denote sets; their cardinalities are counts. In the single-client scaling rules, $C(t) = |C_u(t)|$ and $N_{\text{sub}}(t) = |\mathcal{K}_u(t)|$ are count shorthands with the fixed client index suppressed. They are never used as sets. Figure labels and algorithm steps may omit (t) when all quantities refer to the same decision epoch.

4.1 Per-Subscription Queues

Each subscription has a queue that stores messages waiting to be delivered and processed. Let $Q_{u,k}(t)$ be the queue of subscription $k \in \mathcal{K}_u(t)$, and let

$$b_{u,k}(t) = |Q_{u,k}(t)| \tag{2}$$

be the backlog for that subscription. This is the basic backlog measure used by the client. At the pod level, backlog is a server-owned aggregate. Let $\mathcal{R}_i(t)$ denote the set of subscription queues currently served by pod i, and let $\tilde{b}_{i,r}(t)$ be the backlog of server-side subscription queue $r \in \mathcal{R}_i(t)$. If queue r corresponds to client u and subscription k, then $\tilde{b}_{i,r}(t)$ is the server-side counterpart of $b_{u,k}(t)$; however, $\mathcal{R}_i(t)$ also contains queues belonging to other clients and is maintained by the pod, not by any single client.

Let $z_{u,k,c}(t) \in \{0, 1\}$ indicate whether subscription k of client u is assigned to connection $c \in C_u(t)$. Feasible assignment requires

$$\sum_{c \in C_u(t)} z_{u,k,c}(t) = 1, \qquad \forall u,\ k \in \mathcal{K}_u(t). \tag{3}$$

Let $i_{u,c}(t) \in \mathcal{S}$ denote the pod index reached by connection c. This endpoint mapping differs from the Pod name p_i and from the selected DQL action $a_u(t)$. The number of subscriptions and cumulative subscription backlog on connection c are

$$N_{u,c}(t) = \sum_{k \in \mathcal{K}_u(t)} z_{u,k,c}(t), \tag{4}$$

$$G_{u,c}(t) = \sum_{k \in \mathcal{K}_u(t)} b_{u,k}(t) z_{u,k,c}(t). \tag{5}$$

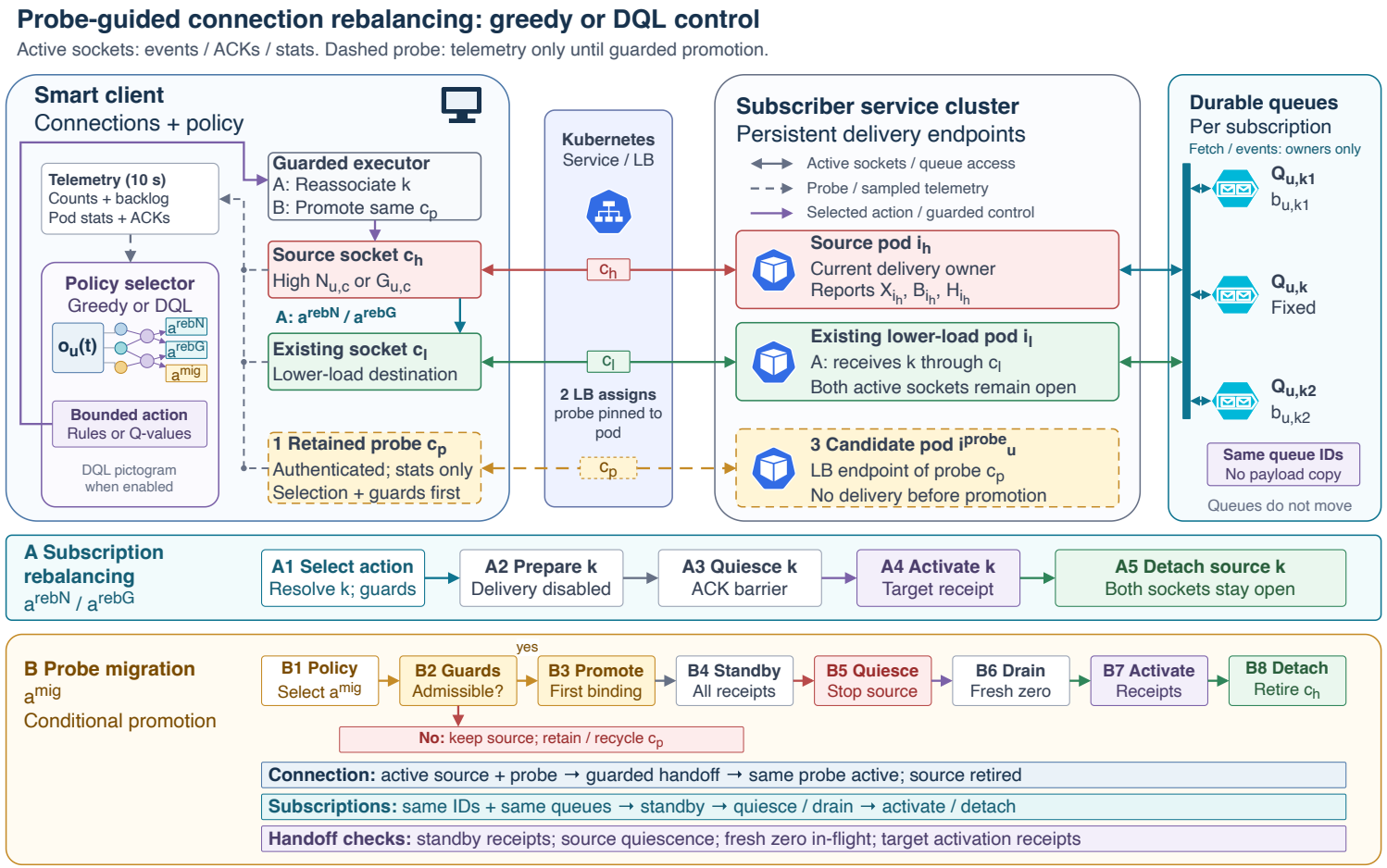


Fig. 1. Shared probe-guided architecture with greedy or DQL selection. Path A rebalances subscriptions between existing sockets. Path B promotes the retained probe through its first standby binding, waits for all standby receipts, quiesces and drains the source, activates the target, detaches the source bindings, and retires the old socket. Unselected sockets and durable queues remain in place. The DQL pictogram denotes the optional learned selector, not an additional handoff service.

The first term captures subscription-management pressure on the connection; the second captures queue pressure induced by the subscriptions mapped to that connection.

4.2 Pod Statistics and Partial Observability

Each pod exposes a statistics tuple through active connections and probe connections:

$$y_i(t) = \big(p_i,\ X_i(t),\ B_i(t),\ H_i\big), \tag{6}$$

Here p_i is the pod name, and $X_i(t)$ is its subscription count. The value $B_i(t)$ is the cumulative backlog across all subscription queues served by pod i. The value H_i is the configured maximum processing threshold. Using the server-side queue set $\mathcal{R}_i(t)$, the pod-level subscription count and cumulative pod backlog are

$$X_i(t) = |\mathcal{R}_i(t)|, \qquad B_i(t) = \sum_{r \in \mathcal{R}_i(t)} \tilde{b}_{i,r}(t). \tag{7}$$

Thus $B_i(t)$ is not defined by summing over the connection set $C_u(t)$ of a particular client. A client's connections determine which pod statistics it can observe and which subscriptions it can move, but the pod backlog itself is the cumulative backlog over all subscription queues served by that pod. These pod-level values include traffic from many clients and are not directly computable by a single client unless reported by the pod.

At each statistics interval Δ (10 seconds in the target deployment), every active connection receives the tuple of its current pod. In addition, client u maintains one authenticated probe connection whose endpoint is selected by the load balancer and receives its tuple. The client retains the probe socket and its endpoint identity until promotion, expiry, or recycling; it does not resample a different pod merely by reading the next observation. Therefore the client observes only a subset $O_u(t) \subseteq \mathcal{S}$ of pod states:

$$O_u(t) = \{i_{u,c}(t) : c \in C_u(t)\} \cup \{i_u^{\text{probe}}(t)\}. \tag{8}$$

The client stores the newest tuple for each observed pod in a local cache. This makes the placement problem partially observable: global cluster variance is available to an offline evaluator or simulator, but not to an individual client controller. In particular, an admissible migration target must be represented by a currently retained probe socket; the client cannot replace the sampled candidate with an unobserved globally optimal pod.

Assumption 1 (Available deployment signals). *For every pod i observed by client u, where $i \in O_u(t)$, the protocol supplies $(p_i, X_i(t), B_i(t), H_i)$ in a statistics frame, header, or control message. Client u can also measure its own per-subscription backlogs $b_{u,k}(t)$, delivered message count, processed message count, acknowledged message count, active connection count, and migration count over each interval Δ.*

Assumption 2 (Transport abstraction). *The underlying transport is required only to maintain long-lived client–pod connections, multiplex one or more subscriptions over a connection, support subscription migration or reassignment, and expose periodic statistics through headers, frames, or control messages. STOMP-over-TCP, custom TCP streams, and WebSocket-based messaging are concrete examples of this abstraction.*

Remark 1. *The pod threshold H_i is not treated as an actual drain rate. The true drain rate depends on the queue subsystem, pod scheduling, network transport, connection behavior, client processing capacity, and acknowledgement rate. Hence $B_i(t)/(H_i\Delta + \varepsilon_0)$ is used only as a dimensionless pod-side pressure ratio. Here B_i counts messages, H_i has units of events per second, and ε_0 has units of messages. The ratio is not an estimate of the actual drain time.*

4.3 Local Imbalance Measures

For client u, define the mean subscription count and mean backlog load across its own connections:

$$\bar{N}_u(t) = \frac{1}{|C_u(t)|} \sum_{c \in C_u(t)} N_{u,c}(t), \tag{9}$$

$$\bar{G}_u(t) = \frac{1}{|C_u(t)|} \sum_{c \in C_u(t)} G_{u,c}(t). \tag{10}$$

The corresponding variances are

$$V_u^N(t) = \frac{1}{|C_u(t)|} \sum_{c \in C_u(t)} \big(N_{u,c}(t) - \bar{N}_u(t)\big)^2, \tag{11}$$

$$V_u^G(t) = \frac{1}{|C_u(t)|} \sum_{c \in C_u(t)} \big(G_{u,c}(t) - \bar{G}_u(t)\big)^2. \tag{12}$$

The first variance captures subscription-management skew, while the second captures backlog skew. The local rebalancing objective is therefore

$$V_u(t) = \eta_N \widehat{V}_u^N(t) + \eta_G \widehat{V}_u^G(t), \qquad \eta_N + \eta_G = 1, \tag{13}$$

where hats denote normalized quantities. This formulation is necessary because two connections can have equal backlog but very different subscription counts, or equal subscription counts but very different backlog. The normalized terms place the two variances on a common scale before they are combined. In the evaluation setup, we use positive reference values V_{ref}^N and V_{ref}^G and compute

$$\widehat{V}_u^N(t) = \min\left\{\frac{V_u^N(t)}{V_{\text{ref}}^N}, 1\right\},$$

$$\widehat{V}_u^G(t) = \min\left\{\frac{V_u^G(t)}{V_{\text{ref}}^G}, 1\right\}.$$

Thus $V_u(t)$ is a dimensionless connection-imbalance score in which the subscription-count component and the backlog-load component are comparable. The weights $\eta_N, \eta_G \geq 0$ determine how much the controller prioritizes subscription-management fairness versus backlog-load fairness. For example, larger η_N makes the controller more sensitive to connections carrying too many subscriptions, while larger η_G makes it more sensitive to connections accumulating more queued messages. The constraint $\eta_N + \eta_G = 1$ makes $V_u(t)$ a convex combination rather than an unbounded sum.

4.4 Pod Pressure and Deployable Reward

For an observed pod $i \in O_u(t)$, define normalized pressure terms

$$P_i^X(t) = \min\left\{\frac{X_i(t)}{X_{\text{safe}}}, 1\right\}, \tag{14}$$

$$P_i^B(t) = \min\left\{\frac{B_i(t)}{B_{\text{ref}}}, 1\right\}, \tag{15}$$

$$P_i^H(t) = \min\left\{\frac{B_i(t)}{H_i\Delta + \varepsilon_0}, 1\right\}, \qquad \varepsilon_0 > 0. \tag{16}$$

Here X_{safe} is the subscription count at which pod reliability begins to degrade. The value X_{hard} is the hard reference count at which we treat the pod as severely overloaded by subscription-management work. The value B_{ref} is the backlog

reference used for normalization. The condition $X_{\text{hard}} > X_{\text{safe}}$ is required. Since high subscription count has been observed to make connections unreliable, we also define a nonlinear reliability-risk penalty

$$P_i^R(t) = \left[\frac{X_i(t) - X_{\text{safe}}}{X_{\text{hard}} - X_{\text{safe}}}\right]_+^2, \qquad [x]_+ = \max\{x, 0\}. \tag{17}$$

The pod pressure score is

$$\begin{aligned} P_i(t) =& \omega_X P_i^X(t) + \omega_B P_i^B(t) \\ &+ \omega_H P_i^H(t) + \omega_R P_i^R(t), \\ &\sum_{q\in\{X,B,H,R\}} \omega_q = 1, \qquad \omega_q \geq 0. \end{aligned} \tag{18}$$

Here $P_i^X(t)$ measures normalized subscription-count pressure. The term $P_i^B(t)$ measures normalized cumulative-backlog pressure. The term $P_i^H(t)$ measures backlog pressure relative to the processing threshold. Finally, $P_i^R(t)$ measures nonlinear reliability risk and becomes active only when $X_i(t) > X_{\text{safe}}$. The coefficients $\omega_X, \omega_B, \omega_H, \omega_R$ specify how much importance the policy assigns to each pressure component. For example, if subscription-heavy pods are known to become unreliable before backlog becomes large, then ω_X and ω_R receive greater weight than ω_B and ω_H. If queue accumulation is the dominant failure mode, ω_B and ω_H receive greater weight. This score keeps subscription count and backlog count as distinct load dimensions.

Client u also computes local backlog and backlog-growth pressure:

$$B_u(t) = \sum_{k\in\mathcal{K}_u(t)} b_{u,k}(t), \tag{19}$$

$$\text{Grow}_u(t) = \left[B_u(t) - B_u(t-1)\right]_+, \qquad t \geq 1. \tag{20}$$

At initialization, $\text{Grow}_u(0) = 0$. Let $T_u(t)$ be the effective client-side throughput in events per second. The rates $T_u^{\text{del}}(t)$, $T_u^{\text{proc}}(t)$, and $T_u^{\text{ack}}(t)$ are the delivered, processed, and acknowledged counts in the measured interval, each divided by its duration Δ:

$$T_u(t) = \min\{T_u^{\text{del}}(t), T_u^{\text{proc}}(t), T_u^{\text{ack}}(t)\}. \tag{21}$$

For an action that selects or keeps pod i, the deployable reward is

$$\begin{aligned} R_u(t) =& -\lambda_1 \widehat{B}_u(t) - \lambda_2 \widehat{\text{Grow}_u}(t) - \lambda_3 P_i(t) - \lambda_4 V_u(t) \\ &- \lambda_5 M_u(t) - \lambda_6 \widehat{C}_u(t) + \lambda_7 \widehat{T}_u(t), \end{aligned} \tag{22}$$

where $M_u(t)$ is the migration indicator or normalized migration count, $\widehat{C}_u(t) = |C_u(t)|/C_{\max}$, and $\lambda_\ell \geq 0$ with $\sum_{\ell=1}^7 \lambda_\ell = 1$. The hats on backlog, backlog growth, and throughput denote their normalized versions under the stated experiment's scaling convention, not different physical quantities. This reward is computable by the client from local measurements and observed pod tuples.

Detailed reference-scale calibration, the global evaluation objective, and step-by-step proofs of the subscription-density and backlog-variance properties are provided in Section A of the Online Supplementary Material. The full notation reference is in Section B of the Online Supplementary Material.

5 Policy Selection and Guarded Rebalancing

The framework places a policy selector above three concrete runtime mechanisms: pod migration, client-side connection scaling, and subscription redistribution across active connections. These mechanisms solve different parts of the same load-distribution problem. Migration changes where a connection terminates, scaling changes how much parallelism the client maintains, and rebalancing changes how subscriptions use that parallelism. Keeping the mechanisms separate makes each action deployable, while the selected policy decides which action is appropriate at a given control epoch.

The separation between reward and feasibility is deliberate. Safe-RL research distinguishes performance optimization from constraint satisfaction [18]; a migration penalty alone cannot enforce ownership or readiness. Krasowski et al. distinguish action masking, replacement, and projection as intervention mechanisms [26]. Our masks and execution-time checks restrict the deployable actions, but do not confer the formal safety guarantees of a verified environment model.

5.1 Greedy and Learned Policy Roles

Both policy families use the client-visible measurements and deterministic execution checks defined above. In the scale-out study, the guarded-greedy selector requests a^{mig} whenever that action is admissible and otherwise stays. Its guard assesses projected source and target pressure after moving the selected subscription group, so a lightly subscribed but heavily backlogged candidate need not be accepted. The source-selection and pressure checks are shared with DQL; they are not learned from DQL's outcomes.

The mixed-workload study uses a different deterministic comparator, labelled *Heuristic*: it applies threshold rules for subscription-count and backlog imbalance as well as migration. The independent *Scripted* timing controls use pressure-unaware schedules and are sensitivity controls, not substitutes for the stronger guarded-greedy comparison. Table 2 keeps these identities distinct.

Table 2. Policy roles and the comparison each supports. Greedy and DQL share the rebalancing mechanism; the studies retain their original policy labels.

Policy	Decision rule	Evidence role
Guarded-greedy	Promote an admissible pressure-improving probe	Strong non-learning scale-out control
Heuristic	Threshold rules over count, backlog, and placement	Non-learning multi-objective comparator
Frozen DQL	Masked maximum learned action value; no evaluation updates	Learned multi-objective selector
Scripted schedules	Fixed or jittered timing; random source and target	Pressure-unaware sensitivity controls
Static / scale-out only	Retain established connection assignment	No-rebalancing reference

The shared contribution is probe-guided actuation, not the invention of either greedy search or Q-learning. The experiments compare policy choices without feeding DQL's holdout decisions back into the scripted rules. Learning traces can inform separate development work, but automatic DQL-to-script tuning is not part of the evaluated system.

5.2 DQL State, Action, and Value Function

The client-side controller is modeled as a partially observable sequential decision problem. At each control epoch, client u constructs an observation vector

$$o_u(t) = \left[o_u^B(t), o_u^N(t), o_u^G(t), o_u^C(t), o_u^P(t), o_u^D(t) \right], \tag{23}$$

where $o_u^B(t)$ contains normalized local backlog and backlog growth. The branch $o_u^N(t)$ describes subscription-count imbalance using terms such as $N_{u,c}(t)$ and $V_u^N(t)$. The branch $o_u^G(t)$ describes backlog imbalance using terms such as $G_{u,c}(t)$ and $V_u^G(t)$. The branch $o_u^C(t)$ contains connection count and effective throughput. The branch $o_u^P(t)$ contains pressure, subscription count, and cumulative backlog for the current and probe pods. Finally, $o_u^D(t)$ contains cooldown and statistics-staleness features. For current-pod features, $c^\star$ denotes the connection being evaluated. All quantities are normalized before they enter the Q-network.

The DQL action space is defined using bounded action templates rather than all possible subscription assignments, which keeps the decision problem tractable:

$$\mathcal{A} = \{a^{\text{stay}}, a^{\text{reb}N}, a^{\text{reb}G}, a^{\text{scale+}}, a^{\text{scale}-}, a^{\text{mig}}\}. \tag{24}$$

Here a^{stay} makes no change. The action $a^{\text{reb}N}$ moves one subscription from the connection with the most subscriptions to the one with the fewest. The action $a^{\text{reb}G}$ moves the subscription with the largest predicted decrease in backlog variance. The action $a^{\text{scale+}}$ opens one new connection and, when allowed, makes bounded subscription moves toward the target Θ_N. The action $a^{\text{scale}-}$ closes a subscription-free connection only if the remaining connections meet the required capacity and observed backlog is zero. The action a^{mig} requests promotion of the retained probe. It transfers the evaluated connection or bounded subscription group only if the admissibility and hysteresis conditions pass. The DQL network selects the action template; it does not choose an arbitrary Pod or directly change a subscription assignment. The deterministic executor computes the concrete source connection, target connection, and subscription using the load-aware rules in Algorithms S1 of the Online Supplementary Material and S2 of the Online Supplementary Material, and may reject an inapplicable template as a guarded no-op. Here Θ_N is the configured maximum desired subscription count per connection, not the network parameter vector θ. The evaluated legacy checkpoint uses a compatible action profile; both scaling actions are disabled in the loaded cluster trial. The later cap-restoring scale-up contract is distinct. Table S13 of the Online Supplementary Material retains these version boundaries and their evaluation scope. DQL uses two action-value networks with the same architecture but different roles. The online network $Q_\theta(o, a)$ is the trainable network used for current action evaluation and epsilon-greedy action selection. The target network $Q_{\theta^-}(o, a)$ is a delayed copy of the online network. It is not used for selecting the live action and is not updated by every gradient step; it is used only to compute a stable Bellman target for the next observation. For a transition $\xi_t = (o_u(t), a_u(t), R_u(t), o_u(t+1))$ sampled from replay buffer $\mathcal{D}$, the target is

$$y_t = R_u(t) + \gamma \max_{a' \in \mathcal{A}_u^{\text{adm}}(t+1)} Q_{\theta^-}(o_u(t+1), a'), \tag{25}$$

where $\gamma \in [0, 1)$ is the discount factor and θ^- denotes the target network parameters. The set $\mathcal{A}_u^{\text{adm}}(t+1) \subseteq \mathcal{A}$ contains the actions admitted by the next observation's execution guards; an infeasible action cannot supply the bootstrap value. The equation applies to non-terminal transitions. At a true terminal state, $y_t = R_u(t)$. An interrupted observation window is not assigned an artificial terminal reward. The online network parameters θ are trained by minimizing

$$\mathcal{L}(\theta) = \mathbb{E}_{\xi_t \sim \mathcal{D}} \left[(y_t - Q_\theta(o_u(t), a_u(t)))^2 \right]. \tag{26}$$

Every K_{sync} training updates, the target network is synchronized by copying the online parameters, $\theta^- \leftarrow \theta$. This separation prevents the same rapidly changing network from defining both the prediction $Q_\theta(o_u(t), a_u(t))$ and the target y_t. The parameters θ and θ^- contain all network weights and biases; Θ_N is instead a subscription-count threshold. The learning rate α controls the update $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta)$ and is distinct from the imbalance weights η_N, η_G. This

formulation makes rebalancing a sequential control problem: a migration, new connection, or subscription move can reduce future backlog but may incur immediate migration or connection cost.

Table 3. Configuration of the v3 DQL policy used in the reported holdout, scale-out simulator, and October live trial. Later learner variants are reported separately in Appendix L of the Online Supplementary Material.

Setting	Value
Network; optimizer	Dense 21–32–6; ReLU hidden, linear output; mini-batch gradient descent
Discount; learning rate	$\gamma = 0.99$; $\alpha = 0.001$
Replay; batch; target copy	Uniform replay, capacity 5,000; batch 16; copy every 250 updates
Bootstrap; loss	Target-network maximum over admitted actions; squared TD error
Training budget	10,000 transitions per candidate; 9,985 updates after batch warm-up
Training exploration	$\epsilon_0 = 1$, multiplier $\exp(\log(0.05)/10{,}000)$; floor 0.05
Reward weights $(\lambda_1, \ldots, \lambda_7)$	$(0.20, 0.15, 0.20, 0.15, 0.10, 0.05, 0.15)$
Pressure weights $(\omega_X, \omega_B, \omega_H, \omega_R)$	$(0.30, 0.25, 0.20, 0.25)$
Imbalance weights	$\eta_N = \eta_G = 0.5$
Frozen evaluation	$\epsilon = 0$; no model or replay updates; checkpoint hash retained

5.3 Shared Control Loop

Algorithm 1 summarizes the policy-independent runtime. The full sizing and rebalancing executors and the DQL training loop are retained in Section C of the Online Supplementary Material.

Algorithm 1: Probe-Guided Selection and Guarded Execution

```
Input: Active connections, retained probe, declared policy and action profile
while client is active do
    Read active/probe telemetry and client queue/ACK counters;
    Construct observation and freshness, capacity, cooldown and handoff masks;
    Select an admitted action using the declared greedy or DQL policy;
    Recheck guards before applying the selected action;
    if migration is selected and admitted then
        Prepare standby bindings on the same probe; await receipts;
        Quiesce the source; wait for fresh zero-in-flight telemetry;
        Activate target bindings; detach source bindings; retire the source socket;
        Retain unselected sessions and replenish the empty probe;
    else
        Execute the admitted bounded rebalancing/scaling action, or stay;
    Record the applied action, observed reward and guard outcome;
    Update learning state only in a declared training run with a valid transition;
```

5.3.1 Exploration, Hysteresis, and Migration. Let $\mathcal{A}_u^{\mathrm{adm}}(t) \subseteq \mathcal{A}$ contain only the templates that pass the current freshness, capacity, cooldown, and handoff guards. The DQL policy uses epsilon-greedy exploration over this masked set:

$$a_u(t) = \begin{cases} \mathrm{Uniform}(\mathcal{A}_u^{\mathrm{adm}}(t)), & \text{w.p. } \epsilon, \\ \arg\max_{a \in \mathcal{A}_u^{\mathrm{adm}}(t)} Q_\theta(o_u(t), a), & \text{w.p. } 1-\epsilon. \end{cases} \tag{27}$$

The exploration probability decreases during training. Frozen validation and deployment policies use $\epsilon = 0$. This prevents unreviewed exploratory changes during evaluation. Online exploration would require a separate safety study. Migration is executed only when the selected action is a^{mig} and the retained probe candidate improves the estimated value by a hysteresis threshold:

$$Q_\theta(o_u(t), a^{\text{mig}}) > Q_\theta(o_u(t), a^{\text{stay}}) + \delta. \tag{28}$$

This threshold prevents oscillatory migrations caused by small or stale statistical differences. It does not choose the probe endpoint: the Kubernetes load balancer already fixed that endpoint when the probe socket was opened. Passing this value test is necessary but not sufficient. The executor also checks telemetry freshness, target capacity, source–target separation, cooldown, and the absence of a conflicting handoff. As a result, most probes can remain telemetry-only and may expire or be recycled without carrying a subscription.

The exact network input order, normalization constants, and deterministic comparator thresholds are retained in Section D of the Online Supplementary Material. Checkpoint persistence and full execution details are in Section C of the Online Supplementary Material.

6 Oracle Cloud Cluster Deployment Experiment

We deployed the connection-replacement mechanism in an Oracle Cloud cluster running Kubernetes in a development environment and first exercised it with 20,000 registered subscriptions. This initial experiment measures live probe promotion, subscription reassociation, handoff timing, and post-migration topology. The controlled scale-out and policy comparisons in Section 7 use a simulator and are reported separately. Sections H of the Online Supplementary Material and 6.4 report separate isolated trials, including selective DQL migration during delivery and a one-to-four-Pod scale-out sequence.

6.1 Deployment and Execution

The deployment contained ten subscriber-service pods. Ten client processes each maintained five active persistent connections, with 400 subscriptions per connection, and one additional subscription-free probe. The inventory had four event-type groups, each with 5,000 subscriptions. Every client loaded the same frozen DQL checkpoint, with exploration disabled and no training updates during evaluation. The deployed action profile admitted rebalancing and migration but masked connection scale-up and scale-down.

Each client could execute at most one guarded connection migration. A selected probe was promoted only after telemetry and execution checks passed. The executor then reassociated the source connection's 400 subscriptions, drained in-flight delivery, retired the source socket, and opened a fresh subscription-free probe. Subscription receipts and server-reported counts were checked against the client assignment. After execution, each process held its topology for 1,800 seconds and checked it again before shutdown. The statistical unit is one deployment run, containing ten client migrations.

6.2 Observed Handoff and Topology

Table 4 shows that all ten clients replaced one active connection and moved its full subscription set. In every case, the destination was the retained probe, the other four active connections remained in place, and the replacement probe had no subscriptions. The final checks confirmed all 2,000 subscriptions and five active connections in each client after the

Table 4. Measured Oracle Cloud cluster deployment results. Handoff and drain statistics summarize ten executed connection migrations. Drain wait is part of handoff time, not an additional independent interval.

Measurement	Observed value
Subscriber-service pods / client processes	10 / 10
Registered and attached subscriptions	20,000
Active persistent connections / probes	50 / 10
Executed full-connection migrations	10
Subscriptions moved per migration / total	400 / 4,000
Handoff time: minimum / median / maximum (s)	7.898 / 14.498 / 101.563
Drain wait: minimum / median / maximum (s)	1.264 / 7.347 / 92.450
Exact final topologies after 1,800 s	10/10
Recorded deliveries / processed and acknowledged	114,986 / 113,149
Processing failures / ACK failures / NACK failures	0 / 0 / 0
Recorded duplicate candidates	0
Independent pod samples / collection errors	310 / 0
Pod restart increase during observation	0

30-minute hold. Thus, 4,000 subscriptions changed their transport association while the complete 20,000-subscription client population remained accounted for.

Handoff duration ranged from 7.898 to 101.563 seconds, with a median of 14.498 seconds. The corresponding source-drain waits ranged from 1.264 to 92.450 seconds, with a median of 7.347 seconds. The range is important: replacement time includes waiting for outstanding work, not just opening a socket. This run deliberately used eight processing workers and a two-second processing delay per event in each client. Those settings exercise handoff under a bounded client drain rate. The endpoint is handoff behavior under this processing limit, rather than maximum Pod throughput.

The paired handoff/drain distributions and their statistical interpretation are retained in Section G of the Online Supplementary Material. An independent collector retained 31 observations of each of the ten pods over approximately 2,422 seconds. All pods remained Ready, with unchanged pod, image, and process identities and no restart increase. These observations establish operational continuity over this observation period. Long-term availability is outside the measurement window.

6.3 Event Accounting and Interpretation

The clients recorded 114,986 deliveries and 113,149 processed and acknowledged events, with no recorded processing, acknowledgement, or negative- acknowledgement failures. The final difference is 1,837 deliveries whose eventual status is not resolved by these counters. Zero duplicate candidates describes the configured observation window, rather than an exactly-once delivery guarantee. The run consumed existing queued work without a matched static-policy workload. Its contribution is measured probe-based connection replacement with 20,000 subscriptions: subscription reassociation, source retirement, and the connection layout after the hold period. Its endpoints are handoff execution and topology retention, not a comparative backlog-drain rate.

The private evidence archive retains the frozen-policy and inventory hashes, per-client decision and topology records, delivery counters, and independent pod observations. A recursive manifest covers 134 files; aggregate values in Table 4 come from the active-execution gate and pod-observation summaries, not from simulator output.

6.4 Loaded DQL Migration During Pod Scale-Out

We next tested selective connection replacement during event delivery in a separate Oracle Cloud cluster run. One client attached 1,000 registered subscriptions to four persistent connections on one subscriber Pod, with 250 subscriptions per connection. The isolated deployment then grew from one to four Ready Pods while the same client process remained running. The second Pod became Ready before standby activation was released; the third and fourth Pods were added during observation. The transport load balancer assigned each empty probe to a Pod. The client used the retained socket only when DQL selected migration and the execution guards passed.

The frozen checkpoint had 9,985 training updates and used $\epsilon = 0$. The trial allowed three executed actions in a configured 600-second window, with a 60-attempt probe-discovery budget. Telemetry had to satisfy the 180-second age limit and 0.8 minimum backlog coverage. Each Pod requested 1.5 CPU cores and 4 GiB memory, with limits of three cores and 6 GiB. The client used 32 workers and a fixed one-second processing delay to keep work outstanding during handoff. This setting tests acknowledgement-aware migration under a bounded client drain rate, rather than maximum Pod capacity.

Two bounded batches each published four synthetic source events. Each event matched 5,000 registered subscriptions, including the client's 1,000-subscription cohort. Thus, publication exposed the queue to 40,000 copies while the measured cohort accounted for 8,000 delivery attempts. Capacity was checked before each batch against the existing 400 MB limit, retaining an 80 MB reserve and prior unreconciled publication costs. No queue purge or quota increase was used.

Table 5. Executed frozen-DQL handoffs in the loaded Oracle Cloud scale-out run. P1 is the initial Pod; P2 and P3 are added Pods. Drain time is included in total handoff time. These are three actions within one run.

Action	Path	Subscriptions	Other sessions	Drain (s)	Total (s)
H1	P1 to P2	250	3 retained	93.862	97.555
H2	P1 to P2	250	3 retained	8.339	10.881
H3	P2 to P3	250	3 retained	76.166	79.657

Table 5 records actual policy execution, not just a migration recommendation. The three observed differences between the migration and stay Q-values were 1.7126, 1.2919 and 1.8201. Each migration was admissible and selected by the frozen network. Before-and-after session maps show that every handoff preserved the other three active connections. Two original sessions remained unchanged throughout the run. The mechanism therefore replaced selected connections without disconnecting the whole pool.

Figure 2(a) separates moved subscriptions from handoff operations. H3 moves the group already transferred by H1, so the three actions comprise 750 subscription-handoff operations involving 500 distinct subscriptions. The final distribution across the four available Pods is 500, 250, 250 and zero. The maximum client population on one Pod falls from 1,000 to 500, a 50% reduction, without increasing the four active connections. The last Pod remains a probe candidate after the three-action budget is reached. Discovering it does not automatically promote it.

Figure 2(b) uses the confirmed ACK receipt timestamp of each recorded delivery. Its cumulative total reaches 8,000, and each of the 1,000 registered subscription hashes has eight acknowledged delivery records. The ledger contains zero recorded ACK failures, processing failures, duplicate candidates or unresolved deliveries at clean closure. Long drain intervals remain visible: source retirement waits for outstanding work instead of discarding it to shorten the measured handoff. The configured window contains 60 decision records; its actual measurement boundaries span 609.716 seconds, including boundary handling.

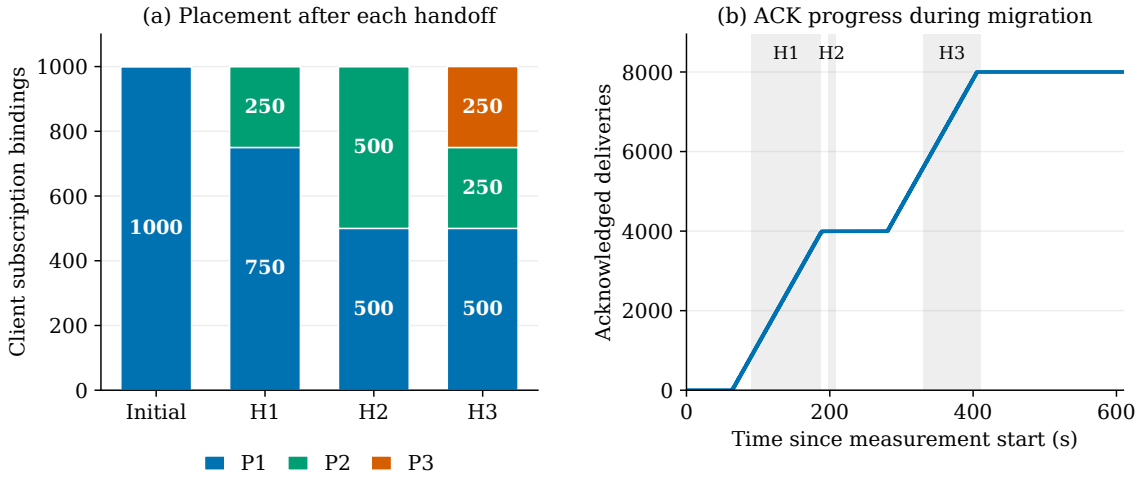


Fig. 2. Measured loaded DQL migration in the Oracle Cloud cluster. (a) Receipt-confirmed client bindings at the initial snapshot and after H1–H3. Each bar accounts for all 1,000 subscriptions; the four active connections each retain 250 subscriptions. P4 has no client bindings and is observed only through a probe. (b) Cumulative ACK receipts, aggregated in one-second bins. Shaded spans show the measured complete handoff intervals, including source drain. The curve describes this bounded workload, not a matched between-policy throughput comparison.

The identity check uses STOMP message identifiers and confirms observed delivery/ACK accounting. Source CloudEvent identities and individual backend queue deletions are outside that ledger's scope. A separate unloaded 1,000-subscription run executed two handoffs in 5.244 and 5.172 seconds; those timings are retained separately because the processing conditions differ. The online and target weights remained identical to the input checkpoint in both runs. The loaded run saved 39 valid experience transitions without applying new training updates. The complete private archive verifies 256 files, and the manuscript figure uses identifier-free aggregates derived from those records. The isolated deployment was restored to one Ready Pod; all 13 shared-resource isolation checks passed.

6.5 TTL Reconnection Comparison

An isolated matched-preload pair used 5,000 subscriptions and 30,000 delivery attempts per arm. Native TTL produced twelve server-matched interruptions and 26,831 confirmed ACKs (89.44%), leaving 3,169 unresolved receipts. The TTL-disabled DQL arm confirmed all 30,000 ACKs without interruptions, but its telemetry guards held every decision at *stay*. A 200-second minimum telemetry sweep exceeded the 180-second freshness deadline. This pair measures expiry-related interruption and receipt completion, not a learned-migration throughput advantage. Unresolved receipts are not classified as event loss.

Detailed TTL implementation paths, recovery endpoints, the two-burst trial, and matched-preload diagnostics are provided in Section I of the Online Supplementary Material. The separate 2,000-subscription calibration and startup-under-load trial is retained in Section H of the Online Supplementary Material.

7 Experimental Method and Locked Simulator Evidence

We evaluate recovery after subscriber-pod scale-out using 20,000 subscriptions, ten persistent connections, and 775 million queued events. The primary comparison measures the use of added pod capacity with guarded-greedy and DQL-guided migration against retaining the initial connection placement. Supporting studies examine multi-objective control, action selection, and adverse probe promotion.

7.1 Recovery After Subscriber-Pod Scale-Out

The primary recovery experiment models the capacity increase from one subscriber pod to four. Ten persistent TCP connections are established on the original pod, carrying 20,000 subscriptions. At the measurement boundary, three additional ready pods provide processing capacity, while the ten established connections retain their original endpoints. The comparison therefore measures how effectively the client uses the added capacity through probe promotion and subscription reassociation.

Four equal subscription groups begin with 100,000, 50,000, 5,000, and zero queued events per subscription, respectively. The initial backlog is $B(0) = 775{,}000{,}000$ events. Round-robin assignment gives each connection 2,000 subscriptions and 77.5 million queued events. Consequently, $V_u^N(0) = V_u^G(0) = 0$: the initial bottleneck is concentrated pod placement. This setup isolates whole-connection migration from pre-existing imbalance between the client's connections.

The model assigns each pod a processing threshold of 100,000 events/s and the client a base acknowledgement capacity of 250,000 events/s. Its bounded connection-parallelism factor is three at ten connections. Service is limited by both client capacity and each connection's share of its pod capacity. The simulator's pressure-dependent service factor is $\max\{0.35, 1 - 0.42P_i^{\text{sim}}\}$, where $P_i^{\text{sim}} \in [0, 1]$ is the simulator's clipped load score, distinct from the deployable pressure in (18). These numerical rates define the simulator's service assumptions rather than measured Pod benchmarks. Pressure is sampled before the action and refreshed after the interval. For the clipped score, this service multiplier lies in $[0.58, 1]$: the model represents pressure-degraded service, not complete Pod service collapse. Arrival rates and processing rates are converted to event counts using the declared $\Delta = 10$ seconds. The drain phase introduces no new arrivals and lasts 120 intervals, or 20 minutes.

We compare *Frozen DQL*, *Kubernetes scale-out only*, and *Scripted probe migration*. All three receive the same capacity increase. The second policy retains the existing connection assignment. The third promotes every candidate admitted by the migration guard and evaluates the framework's deterministic guarded-greedy policy. DQL uses the previously frozen checkpoint without exploration or training. Adaptive policies can replace at most 12 non-improving probes, and actions have a three-interval cooldown. The scripted control uses no learned Q-values: it shares the pressure-aware admission rule and source selection with DQL. We therefore distinguish this *guarded-greedy* policy from the independent ad-hoc schedules below. Its mean throughput is 221,260 events/s, compared with 219,762 for DQL. These are two effective selectors for the shared rebalancing mechanism, not evidence that Q-learning is necessary to use added capacity. The original registered primary contrast remains DQL versus unchanged placement; this broader interpretation does not change that protocol or relabel a post-hoc contrast as confirmatory. An exploratory paired reanalysis of these same twenty seeds gives a greedy-minus-DQL mean throughput difference of 1,498 events/s (pointwise percentile bootstrap 95% interval [362, 2,852], 10,000 resamples). This small rate advantage belongs to the greedy policy in this workload; the mixed-workload study evaluates DQL's separate multi-objective benefit.

Table 6. Persistent-connection recovery after scaling from one to four subscriber Pods. Twenty paired seeds each start with 20,000 subscriptions and 775 million queued events on ten TCP connections pinned to Pod 0. Lower backlog measures are better.

Policy	Backlog AUC	Final backlog (%)	Throughput (k/s)	Active Pods	Migrations
Frozen DQL	0.8376	65.97	219.8	4.00	7.75
Kubernetes scale-out only	0.9551	91.02	58.0	1.00	0.00
Scripted probe migration	0.8359	65.74	221.3	4.00	7.75
DQL vs. scale-out only: 12.30% lower backlog AUC; 278.9% higher mean throughput.					

For remaining backlog $B(t)$, the primary endpoint is the mean normalized backlog over the recovery window,

$$\mathrm{AUC}_B = \frac{1}{T}\sum_{t=1}^{T}\frac{B(t-1)+B(t)}{2B(0)}, \qquad T = 120. \tag{29}$$

Lower AUC_B indicates less accumulated queued work. Twenty fresh paired seeds vary probe placement while holding the workload fixed. We use 10,000 paired bootstrap resamples and 100,000 sign-randomization draws. The registered criterion requires the upper 95% confidence bound of the DQL-minus-baseline AUC difference to be below zero and the one-sided p-value to be below 0.05. These intervals describe variation caused by probe placement within this model.

Table 6 shows that DQL raises mean throughput from 58,000 to 219,762 events/s, a factor of 3.79. It reduces backlog AUC from 0.95510 to 0.83759, an improvement of 12.30%. The paired AUC difference is −0.117504 (95% confidence interval $[-0.121210, -0.113345]$; one-sided $p \simeq 10^{-5}$). DQL leaves 65.97% of the initial backlog after 20 minutes, compared with 91.02% for scale-out only, or 27.52% less remaining backlog relative to that baseline. All 20 DQL episodes use all four pods by the end, with a mean of 7.75 connection migrations. The baseline retains all ten connections on the original pod.

Figure 3 connects the endpoint to the recovery process. The steeper DQL curve follows successful probe promotions that place delivery work on additional pods. The nearby scripted-control curve shows that DQL reaches about the same service capacity as guarded scripted migration. Both improve on the unchanged placement, establishing the benefit of migration in this workload rather than a ranking above every controller with the same actions.

Capacity-activation timelines, seed-level violin plots, and independent ad-hoc timing controls are retained in Section J of the Online Supplementary Material. Those controls do not replace the pressure-aware guarded-greedy comparator above.

7.2 Independent Multi-Objective Holdout

The independent holdout compares the frozen DQL policy with the threshold-based Heuristic and Static policies from Table 2. Full study designs, training traces, distributions, and action-contract diagnostics are retained in Section K of the Online Supplementary Material.

Table S14 of the Online Supplementary Material summarizes the evidence-generating runs. In v3, three candidate networks were trained for exactly 10,000 transitions under seeds 101, 211, and 307. Candidate selection used ten fixed validation seeds and required subscription-count rebalancing ($a^{\mathrm{reb}N}$), backlog rebalancing ($a^{\mathrm{reb}G}$), and full connection migration (a^{mig}) in at least 80% of admissible episodes in their intended scenarios. Candidate 0 (seed 101) passed this gate and was frozen before the holdout seed bank was read.

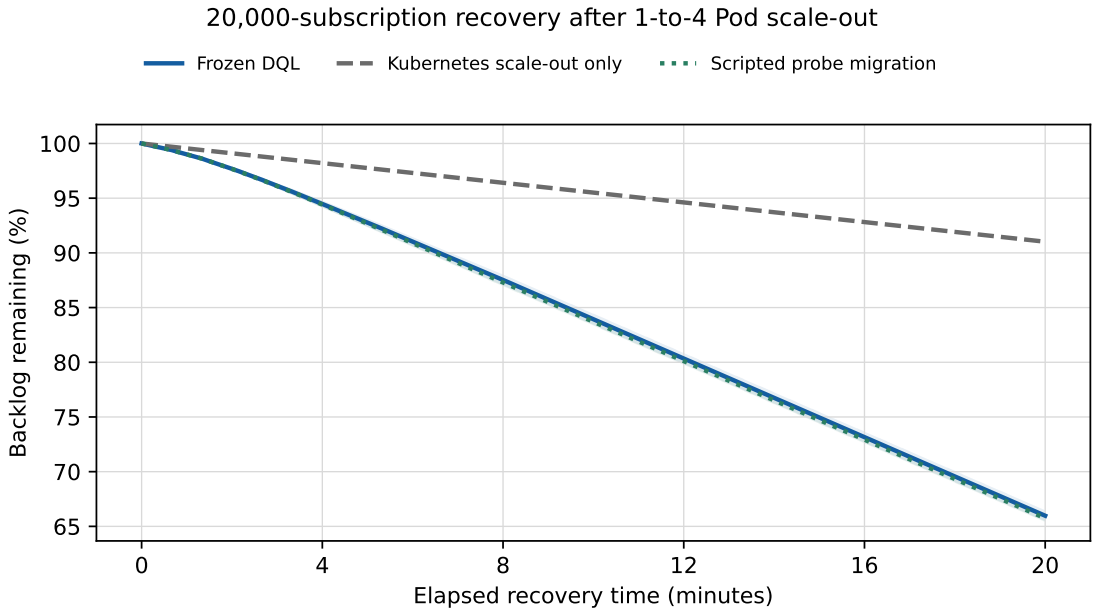


Fig. 3. Backlog recovery after adding three subscriber pods. All policies start with the same 775-million-event backlog. Curves are means over 20 paired seeds; shaded bands are descriptive 95% normal-approximation intervals. The area under each normalized curve defines the primary endpoint.

The independent v3 holdout contains ten previously unused seeds for each of four scenarios and 500 decision intervals per episode. The connection-count skew scenario uses 1,000 subscriptions; the tiered-backlog scenario uses 1,200 subscriptions; and the client-slowdown and pod-pressure-skew scenarios each use 1,600 subscriptions. Every scenario has four modeled pods, four initial persistent connections, and a maximum of six connections. The tiered-backlog scenario assigns 25% of its queues to each initial backlog level 100,000, 50,000, 5,000, and zero. The client-slowdown scenario reduces acknowledgement capacity to 35% of its nominal value between intervals 125 and 375. The other two scenarios isolate initial subscription-count skew and the placement of three of four connections on one pod, respectively.

For policy π, paired episode j, and metric v, let $Y_{\pi,j,v}$ denote the episode mean. We define a direction-adjusted paired difference

$$D_{j,v} = d_v \left(Y_{\mathrm{DQL},j,v} - Y_{\mathrm{base},j,v} \right), \qquad d_v = \begin{cases} +1, & \text{higher is better,} \\ -1, & \text{lower is better.} \end{cases} \tag{30}$$

Thus, a positive $D_{j,v}$ always favors DQL. The relative benefit reported below is

$$\mathrm{RB}_v = 100 \, \frac{\overline{D}_v}{\left| \overline{Y}_{\mathrm{base},v} \right|}. \tag{31}$$

Overbars in these two equations denote means across paired episodes. The metric index v is distinct from the client-population size m. We resample the 40 paired differences 10,000 times to obtain percentile 95% confidence intervals.

Two-sided paired sign-flip tests are exact for the scenario-level sets of ten pairs and use 100,000 Monte Carlo sign vectors for the pooled set of 40 pairs. Pooled p-values are adjusted by the Holm procedure across the complete family of 18 metric-by-baseline tests. The scenario box plots in Figure S16 of the Online Supplementary Material describe quartiles and individual episode outcomes; the paired randomization tests provide the statistical comparison.

We evaluate DQL as a sequential controller through return, operational outcomes, paired effect sizes, and action coverage. Classifier accuracy and ROC–AUC apply to labelled prediction tasks. Applying them here would mainly measure agreement with a heuristic that supplied the correct-action labels, rather than the controller's operating performance.

Across all 40 paired episodes, frozen DQL improves mean reward over the heuristic from −0.1609 to −0.1291. The mean paired difference is 0.0318 (bootstrap 95% CI [0.0191, 0.0455]), equivalent to a 19.8% relative benefit under Eq. (31); the paired standardized effect is $d_z = 0.729$, and the Holm-adjusted sign-flip p-value is below 0.001. Table 7 and Figure S17 of the Online Supplementary Material show that this overall gain comes with tradeoffs.

Table 7. Frozen DQL versus the heuristic on the independent v3 holdout. Positive benefit favors DQL. CIs are paired-bootstrap intervals transformed by Eq. (31); p_H is Holm-adjusted over 18 pooled metric-by-baseline tests.

Metric	DQL	Heuristic	Benefit [95% CI] (%)	p_H
Mean reward	−0.1291	−0.1609	19.8 [11.9, 28.3]	< .001
Backlog ratio	0.1749	0.2002	12.7 [−1.2, 27.1]	.095
Conn.-subscription variance	574.7	5768.7	90.0 [61.0, 120.8]	< .001
Conn.-backlog variance	2.280×10^{13}	1.568×10^{13}	−45.4 [−72.5, −22.6]	.022
Subscription imbalance	0.0898	0.4343	79.3 [59.1, 99.5]	< .001
Backlog imbalance	0.2194	0.1638	−34.0 [−56.4, −13.6]	.022
Combined conn. imbalance	0.1546	0.2991	48.3 [33.8, 63.8]	< .001
Pod pressure	0.1878	0.2051	8.4 [3.6, 13.6]	.022
Throughput ratio	0.2840	0.2973	−4.5 [−7.1, −2.2]	.022

The largest benefit is a 90.0% reduction in connection-level subscription variance, consistent with the design objective of moving subscriptions away from overloaded persistent connections. Subscription imbalance falls by 79.3%, combined connection imbalance by 48.3%, and mean pod pressure by 8.4%. Mean backlog ratio is 12.7% lower, but its interval crosses zero and the adjusted test is not significant ($p_H = .095$). Conversely, DQL has 45.4% higher connection-backlog variance, 34.0% higher normalized backlog imbalance, and 4.5% lower throughput than the heuristic. The scalar reward still favors DQL because its configured penalties place substantial weight on subscription pressure and combined imbalance. The holdout therefore identifies a multi-objective advantage for DQL and a queue-drainage advantage for the heuristic.

7.3 Adverse-Probe Ablation

The separate 30-seed, 1,200-subscription ablation compares the same frozen DQL policy with and without one forced promotion to the highest-pressure probe Pod. Guarded DQL reduces normalized backlog AUC from 0.68938 to 0.66046: a paired difference of −0.028918 (95% CI [−0.039444, −0.019073]), or 4.19%. The guarded heuristic retains the lower AUC of 0.59104. This isolates the benefit of guarding promotion, not universal superiority of learned selection. The complete five-policy comparison, methods, and separate five-seed 20,000-subscription confirmation are retained in Section K.7 of the Online Supplementary Material.

Reviewer artifact access. The accompanying supplementary archive, `reviewer-evidence.zip`, contains episode-level simulator CSVs, declared protocols, original aggregate analyses, the frozen 21–32–6 checkpoint, and identifier-free live summaries. Its README gives the file map and exact offline commands. A Python 3.9+ standard-library script verifies the checksum manifest, checks paired cohort coverage, recomputes policy means and selected effect sizes, and reconciles the loaded trial's reported subscription and ACK totals. A separate inference option evaluates the saved network without transport access or model updates. The archive supports reanalysis and checkpoint inspection; it does not expose corporate credentials, raw event payloads, private endpoint identifiers, or the proprietary service source. It is not a complete independent deployment rerun. Provenance records distinguish unchanged measurements from redacted local-path metadata.

The public research repository is available at https://github.com/manish94u/rebalancePersistance. It provides the manuscript sources, standalone simulator, redacted evidence, frozen checkpoint, and step-by-step replication instructions.

8 Stability Controls and Policy Validation

Because the controller operates with partial and periodically refreshed statistics, stability is enforced at two levels. During learning, replay, target-network synchronization, bounded rewards, and epsilon decay limit the rate at which value estimates change. During execution, feasibility masks, minimum-improvement thresholds, cooldowns, and the receipt-gated handoff protocol prevent a high Q-value from bypassing required operating rules. These controls bound the available transitions. Their role is operational stability, distinct from a proof of learned-policy convergence.

Classical tabular Q-learning guarantees assume a stationary, fully observable Markov decision process and sufficient visitation of every state-action pair. The client instead receives delayed partial observations in an environment whose queue arrivals, acknowledgement capacity, and competing clients can change. Policy quality is therefore evaluated under the stated workloads, without a global-optimality guarantee. The finite v3 traces describe training behavior rather than Bellman-loss convergence. The v4–v6 results distinguish training completion from action coverage. The validation protocol uses action-coverage, fault-safety, churn, and performance gates to assess each frozen policy.

RL evaluation also separates adaptation from regression. Biagiola and Tonella test how RL systems adapt to changed environments while retaining prior capabilities [3]. Patterson et al. discuss performance variation, baseline construction, hyperparameter selection, and experimenter bias in empirical RL [32]. These concerns motivate reporting the training, frozen-policy comparison, and live handoff evidence separately. A successful handoff and a higher comparative reward answer different evaluation questions.

9 Discussion

The main systems finding is that subscriber capacity and client access to that capacity are separate. Adding Pods supplies service capacity; moving existing subscription-bearing connections makes it reachable by persistent clients. The guarded-greedy and DQL policies both achieve this in the scale-out model. Their nearby throughput results support the shared mechanism rather than requiring a claim that learning beats a strong greedy rule in every workload.

Policy choice depends on the operational objective. Greedy control gives a direct, inspectable response to measured pressure. DQL can account for delayed costs and several competing objectives. A subscription move changes future backlog distribution; migration trades immediate handoff cost for later capacity. Both selectors operate above the same bounded, checked actions.

The action-contract studies provide a further design lesson: a useful executor and a policy that reliably selects it are separate requirements. The cap-restoring scale-up preflight improved reward and maximum subscription count, but the expanded learner study did not pass every coverage gate. Appendix L of the Online Supplementary Material retains these results. They explain why the live checkpoint uses a compatible, restricted rebalancing profile.

10 Conclusion

We presented a probe-guided, client-side framework for rebalancing persistent subscription connections without requiring cluster-wide client visibility. The mechanism combines per-subscription queues, active-connection telemetry, and a load-balancer-assigned probe. Greedy rules or DQL select bounded actions, and a shared executor carries out receipt-gated handoff. The technical contribution is the link between partial observations, endpoint-preserving probe promotion, and selective subscription reassociation.

The primary experiment demonstrates recovery after a one-to-four pod capacity increase with 20,000 subscriptions and 775 million queued events. Guarded-greedy migration and frozen DQL deliver 221,260 and 219,762 events/s, respectively, compared with 58,000 events/s for unchanged placement. Frozen DQL uses all four pods, delivers 3.79 times the mean throughput, and leaves 27.52% less backlog after 20 minutes than retaining the ten original connection assignments. Its 12.30% reduction in backlog area satisfies the registered paired statistical criterion. The adverse-probe ablation complements this result: evaluating the candidate before promotion reduces backlog area by 4.19% relative to forcing one harmful migration. The Oracle Cloud experiments establish live handoff execution, subscription retention, and observed delivery behavior. The comparative throughput and backlog-recovery gains are simulator measurements under the stated service assumptions. These results support two evaluated policy options above the same probe-guided handoff mechanism.

References

[1] Abeer Abdel Khaleq and Ilkyeun Ra. 2021. Intelligent Autoscaling of Microservices in the Cloud for Real-Time Applications. *IEEE Access* 9 (2021), 35464–35476. doi:10.1109/ACCESS.2021.3061890

[2] Ratnadeep Bhattacharya, Yuan Gao, and Timothy Wood. 2024. Dynamically Balancing Load with Overload Control for Microservices. *ACM Transactions on Autonomous and Adaptive Systems* 19, 4 (2024), 1–23. doi:10.1145/3676167

[3] Matteo Biagiola and Paolo Tonella. 2022. Testing the Plasticity of Reinforcement Learning-based Systems. *ACM Transactions on Software Engineering and Methodology* 31, 4 (2022), 1–46. doi:10.1145/3511701

[4] Valeria Cardellini, Francesco Lo Presti, Matteo Nardelli, and Gabriele Russo Russo. 2022. Runtime Adaptation of Data Stream Processing Systems: The State of the Art. *Comput. Surveys* 54, 11s (2022), 1–36. doi:10.1145/3514496

[5] Maria Casimiro, Diogo Soares, David Garlan, Luís Rodrigues, and Paolo Romano. 2024. Self-adapting Machine Learning-based Systems via a Probabilistic Model Checking Framework. *ACM Transactions on Autonomous and Adaptive Systems* 19, 3 (2024), 1–30. doi:10.1145/3648682

[6] Luca Caviglione, Mauro Gaggero, Massimo Paolucci, and Roberto Ronco. 2021. Deep reinforcement learning for multi-objective placement of virtual machines in cloud datacenters. *Soft Computing* 25, 19 (2021), 12569–12588. doi:10.1007/s00500-020-05462-x

[7] Genxin Chen, Jin Qi, Ying Sun, Xiaoxuan Hu, Zhenjiang Dong, and Yanfei Sun. 2023. A collaborative scheduling method for cloud computing heterogeneous workflows based on deep reinforcement learning. *Future Generation Computer Systems* 141 (2023), 284–297. doi:10.1016/j.future.2022.11.032

[8] Xing Chen, Lijian Yang, Zheyi Chen, Geyong Min, Xianghan Zheng, and Chunming Rong. 2023. Resource allocation with workload-time windows for cloud-based software services: A deep reinforcement learning approach. *IEEE Transactions on Cloud Computing* 11, 2 (2023), 1871–1885. doi:10.1109/TCC.2022.3169157

[9] Xing Chen, Fangning Zhu, Zheyi Chen, Geyong Min, Xianghan Zheng, and Chunming Rong. 2022. Resource allocation for cloud-based software services using prediction-enabled feedback control with reinforcement learning. *IEEE Transactions on Cloud Computing* 10, 2 (2022), 1117–1129. doi:10.1109/TCC.2020.2992537

[10] Zheyi Chen, Jia Hu, Geyong Min, Chunbo Luo, and Tarek El-Ghazawi. 2022. Adaptive and efficient resource allocation in cloud datacenters using actor-critic deep reinforcement learning. *IEEE Transactions on Parallel and Distributed Systems* 33, 8 (2022), 1911–1923. doi:10.1109/TPDS.2021.3132422

[11] Long Cheng, Yue Wang, Feng Cheng, Cheng Liu, Zhiming Zhao, and Ying Wang. 2024. A deep reinforcement learning-based preemptive approach for cost-aware cloud job scheduling. *IEEE Transactions on Sustainable Computing* 9, 3 (2024), 422–432. doi:10.1109/TSUSC.2023.3303898

[12] Ivan Čilić, Ivana Podnar Žarko, Pantelis Frangoudis, and Schahram Dustdar. 2026. QoS-Aware Load Balancing in the Computing Continuum via Multi-Player Bandits. arXiv preprint arXiv:2512.18915v2. https://arxiv.org/abs/2512.18915v2

[13] Ding Ding, Xiaocong Fan, Yihuan Zhao, Kaixuan Kang, Qian Yin, and Jing Zeng. 2020. Q-learning based dynamic task scheduling for energy-efficient cloud computing. *Future Generation Computer Systems* 108 (2020), 361–371. doi:10.1016/j.future.2020.02.018

[14] Gabriel Dulac-Arnold, Nir Levine, Daniel J. Mankowitz, Jerry Li, Cosmin Paduraru, Sven Gowal, and Todd Hester. 2021. Challenges of real-world reinforcement learning: Definitions, benchmarks and analysis. *Machine Learning* 110, 9 (2021), 2419–2468. doi:10.1007/s10994-021-05961-4

[15] Marios Fragkoulis, Paris Carbone, Vasiliki Kalavri, and Asterios Katsifodimos. 2024. A survey on the evolution of stream processing systems. *The VLDB Journal* 33, 2 (2024), 507–541. doi:10.1007/s00778-023-00819-8

[16] Omid Gheibi and Danny Weyns. 2024. Dealing with Drift of Adaptation Spaces in Learning-based Self-Adaptive Systems Using Lifelong Self-Adaptation. *ACM Transactions on Autonomous and Adaptive Systems* 19, 1 (2024), 1–57. doi:10.1145/3636428

[17] Omid Gheibi, Danny Weyns, and Federico Quin. 2021. Applying Machine Learning in Self-adaptive Systems: A Systematic Literature Review. *ACM Transactions on Autonomous and Adaptive Systems* 15, 3 (2021), 1–37. doi:10.1145/3469440

[18] Shangding Gu, Long Yang, Yali Du, Guang Chen, Florian Walter, Jun Wang, and Alois Knoll. 2024. A Review of Safe Reinforcement Learning: Methods, Theories, and Applications. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46, 12 (2024), 11216–11235. doi:10.1109/TPAMI.2024.3457538

[19] Diego Hortelano, Ignacio de Miguel, Ramón J. Durán Barroso, Juan Carlos Aguado, Noemí Merayo, Lidia Ruiz, Adrian Asensio, Xavi Masip-Bruin, Patricia Fernández, Rubén M. Lorenzo, and Evaristo J. Abril. 2023. A comprehensive survey on reinforcement-learning-based computation offloading techniques in Edge Computing Systems. *Journal of Network and Computer Applications* 216 (2023), 103669. doi:10.1016/j.jnca.2023.103669

[20] Jiaxin Hou, Yingbo Wu, Junpeng Cai, and Zhiwen Zhou. 2023. QoE-guaranteed distributed offloading decision via partially observable deep reinforcement learning for edge-enabled internet of things. *Neural Computing and Applications* 35, 29 (2023), 21603–21619. doi:10.1007/s00521-023-08905-2

[21] Liang Huang, Suzhi Bi, and Ying-Jun Angela Zhang. 2020. Deep Reinforcement Learning for Online Computation Offloading in Wireless Powered Mobile-Edge Computing Networks. *IEEE Transactions on Mobile Computing* 19, 11 (2020), 2581–2593. doi:10.1109/TMC.2019.2928811

[22] Shashikant Ilager, Jakob Fahringer, Alessandro Tundo, and Ivona Brandić. 2026. A Decentralized and Self-Adaptive Approach for Monitoring Volatile Edge Environments. *ACM Transactions on Autonomous and Adaptive Systems* 21, 2 (2026), 1–30. doi:10.1145/3736726

[23] Amanda Jayanetti, Saman Halgamuge, and Rajkumar Buyya. 2022. Deep reinforcement learning for energy and time optimized scheduling of precedence-constrained tasks in edge-cloud computing environments. *Future Generation Computer Systems* 137 (2022), 14–30. doi:10.1016/j.future.2022.06.012

[24] Alireza Kavianifar and Saeed Jalili. 2026. RS-DRL: Managing Uncertainty in Self-Adaptive Systems Based on a Novel Continuous Deep Reinforcement Learning Method. *ACM Transactions on Autonomous and Adaptive Systems* 21, 3 (2026), 1–30. doi:10.1145/3779424

[25] Tahseen Khan, Wenhong Tian, Guangyao Zhou, Shashikant Ilager, Mingming Gong, and Rajkumar Buyya. 2022. Machine learning (ML)-centric resource management in cloud computing: A review and future directions. *Journal of Network and Computer Applications* 204 (2022), 103405. doi:10.1016/j.jnca.2022.103405

[26] Hanna Krasowski, Jakob Thumm, Marlon Müller, Lukas Schäfer, Xiao Wang, and Matthias Althoff. 2023. Provably Safe Reinforcement Learning: Conceptual Analysis, Survey, and Benchmarking. *Transactions on Machine Learning Research* (2023). https://openreview.net/forum?id=mcN0ezbnzO

[27] Hongjian Li, Liang Lu, Wenhu Shi, Gangfan Tan, and Hao Luo. 2023. Energy-aware scheduling for Spark job based on deep reinforcement learning in cloud. *Computing* 105, 8 (2023), 1717–1743. doi:10.1007/s00607-023-01171-z

[28] Zhaoze Liu, Yajuan Sun, Cong Liu, Zhiming Zhao, and Long Cheng. 2026. Data-intensive task scheduling in geo-distributed cloud computing: a deep reinforcement learning approach. *Journal of Cloud Computing* 15, 1, Article 79 (2026), 15 pages. doi:10.1186/s13677-026-00894-6

[29] Andreas Metzger, Clément Quinton, Zoltán Ádám Mann, Luciano Baresi, and Klaus Pohl. 2024. Realizing self-adaptive systems via online reinforcement learning and feature-model-guided exploration. *Computing* 106, 4 (2024), 1251–1272. doi:10.1007/s00607-022-01052-x

[30] Sarita Negi, Devesh Pratap Singh, and Man Mohan Singh Rauthan. 2024. A systematic literature review on soft computing techniques in cloud load balancing network. *International Journal of System Assurance Engineering and Management* 15, 3 (2024), 800–838. doi:10.1007/s13198-023-02217-3

[31] Gorka Nieto, Idoia de la Iglesia, Unai Lopez-Novoa, and Cristina Perfecto. 2024. Deep Reinforcement Learning techniques for dynamic task offloading in the 5G edge-cloud continuum. *Journal of Cloud Computing* 13, 1, Article 94 (2024), 24 pages. doi:10.1186/s13677-024-00658-0

[32] Andrew Patterson, Samuel Neumann, Martha White, and Adam White. 2024. Empirical Design in Reinforcement Learning. *Journal of Machine Learning Research* 25, 318 (2024), 1–63. https://jmlr.org/papers/v25/23-0183.html

[33] Yousef Sanjalawe, Salam Al-E'mari, Salam Fraihat, and Sharif Makhadmeh. 2025. AI-driven job scheduling in cloud computing: A comprehensive review. *Artificial Intelligence Review* 58, 7, Article 197 (2025), 113 pages. doi:10.1007/s10462-025-11208-8

[34] José Santos, Mattia Zaccarini, Filippo Poltronieri, Mauro Tortonesi, Cesare Stefanelli, Nicola Di Cicco, and Filip De Turck. 2025. HephaestusForge: Optimal microservice deployment across the Compute Continuum via Reinforcement Learning. *Future Generation Computer Systems* 166 (2025), 107680. doi:10.1016/j.future.2024.107680

[35] G. Selvakumar, L. S. Jayashree, and S. Arumugam. 2023. Latency Minimization Using an Adaptive Load Balancing Technique in Microservices Applications. *Computer Systems Science and Engineering* 46, 1 (2023), 1215–1231. doi:10.32604/csse.2023.032509

[36] Dawei Sun, Shang Gao, Xunyun Liu, Xindong You, and Rajkumar Buyya. 2020. Dynamic redirection of real-time data streams for elastic stream computing. *Future Generation Computer Systems* 112 (2020), 193–208. doi:10.1016/j.future.2020.05.021

[37] Dawei Sun, Chunlin Zhang, Shang Gao, and Rajkumar Buyya. 2024. An adaptive load balancing strategy for stateful join operator in skewed data stream environments. *Future Generation Computer Systems* 152 (2024), 138–151. doi:10.1016/j.future.2023.11.002

[38] Wenting Wei, Huaxi Gu, Kun Wang, Jianjia Li, Xuan Zhang, and Ning Wang. 2023. Multi-dimensional Resource Allocation in Distributed Data Centers using Deep Reinforcement Learning. *IEEE Transactions on Network and Service Management* 20, 2 (2023), 1817–1829. doi:10.1109/TNSM.2022.3213575

[39] Danny Weyns, Ilias Gerostathopoulos, Nadeem Abbas, Jesper Andersson, Stefan Biffl, Premek Brada, Tomas Bures, Amleto Di Salle, Matthias Galster, Patricia Lago, Grace Lewis, Marin Litoiu, Angelika Musil, Juergen Musil, Panos Patros, and Patrizio Pelliccione. 2023. Self-Adaptation in Industry: A Survey. *ACM Transactions on Autonomous and Adaptive Systems* 18, 2 (2023), 1–44. doi:10.1145/3589227

[40] Mariam Yahya, Alexander Conzelmann, and Setareh Maghsudi. 2024. Decentralized Task Offloading and Load-Balancing for Mobile Edge Computing in Dense Networks. *IEEE Communications Letters* 28, 8 (2024), 1954–1958. doi:10.1109/LCOMM.2024.3416833

[41] Shuhao Zhang, Juan Soto, and Volker Markl. 2024. A survey on transactional stream processing. *The VLDB Journal* 33, 2 (2024), 451–479. doi:10.1007/s00778-023-00814-z

[42] Guangyao Zhou, Wenhong Tian, Rajkumar Buyya, Ruini Xue, and Liang Song. 2024. Deep reinforcement learning-based methods for resource scheduling in cloud computing: A review and future directions. *Artificial Intelligence Review* 57, 5, Article 124 (2024), 42 pages. doi:10.1007/s10462-024-10756-9

[43] Guangyao Zhou, Ruiming Wen, Wenhong Tian, and Rajkumar Buyya. 2022. Deep reinforcement learning-based algorithms selectors for the resource scheduling in hierarchical Cloud computing. *Journal of Network and Computer Applications* 208 (2022), 103520. doi:10.1016/j.jnca.2022.103520